Secure Belief: Exploitation of “Self-Only” Cross-Site Scripting in Google Code

Secure Belief: Exploitation of “Self-Only” Cross-Site Scripting in Google Code - Author not stated, amolnaik4.blogspot.com.

- Published: date not stated
- Original: <https://amolnaik4.blogspot.com/2011/03/exploitation-of-self-only-cross-site.html>
- Preserved from: https://amolnaik4.blogspot.com/2011/03/exploitation-of-self-only-cross-site.html (stored) on 2026-08-06
- Capture timestamp: 20120530172623
- Licence: unknown

Content

> UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

As an attempt to contribute for Google’s Rewarding Web Application Security Research, I started working on Google Code in search of vulnerabilities that could qualify for the reward program. That is where I came across a Cross-site Scripting bug which seems “not exploitable” at first. As Google has patched the vulnerable pages, I’m going to explain the exploitation of this bug here.

“Self-Only” Cross-Site Scripting

“Self-Only” XSS term is referred in past by many researchers for “**CSRF Protected XSS**”. You can read it here and here. The issue I found in Google Code site was not related to “CSRF protected XSS”. I referred this bug as “Self-Only” XSS due to its nature because this was not a GET or POST XSS and was only exploited by the victim. This means that the victim has to type “<script>alert(document.cookie)</script>” in the input box and click “Go!” to get his own cookies. Confused! OK. I’ll try to explain this with Google Code example.

Cross-Site Scripting in Google Code

Google Code hosts the documentation for Google APIs. The Google MAP API documentation includes the examples pages to demonstrate different map functions. One of them is “Simple Geocoding” example. The link for this page is:

http://code.google.com/intl/es-MX/apis/maps/documentation/javascript/v2/examples/geocoding-simple.html

This page displays geo-location of the requested location on the map. The page makes a GET request to Google Map API and displays the result.

[[image: image]](https://lh5.googleusercontent.com/-VI-sJivVszE/TYhEpByokGI/AAAAAAAAAOs/gTolStxIoAE/s1600/main-page.JPG)

When requested with a valid location followed by XSS payload e.g. pune<script>alert(document.cookie)</script>, makes following GET request to Google Map API :

http://map.google.com/maps/geo?output=json&oe=utf-8&q=pune%3Cscript%3Ealert%28document.cookie%29%3C%2Fscript%3E&key=ABQIAAAAzr2EBOXUKnm_jVnk0OJI7xSosDVG8KKPE1-m51RBrvYughuyMxQ-i1QfUnH94QxWIa6N4U6MouMmBA&mapclient=jsapi&hl=en&callback=

Google Map API returns JSON response to the request as below:

[[image: image]](https://lh6.googleusercontent.com/-FZki8IRfM-s/TYhF8xeiDNI/AAAAAAAAAO0/34283q6Mxj4/s1600/json-request.JPG)

This response is rendered by the example page at Google Code as below:

[[image: image]](https://lh4.googleusercontent.com/-YZreh7BbGR4/TYhGahp1H3I/AAAAAAAAAO4/JeSNJpx4UeU/s1600/dom-source.JPG)

And executes the payload in victim's browser:

[[image: image]]
(https://lh6.googleusercontent.com/-8TIJWx8VjlQ/TYhHQaJENOI/AAAAAAAAAO8/fzVAk-JTBA8/s1600/xss-poc.JPG)

This is due to lack of sanitization of malicious data while rendering the output back to the example page.

A quick analysis for request/response reveals that this XSS cannot be exploited by classical GET or POST method. As an attacker, we cannot control any request that can be used to craft payload and when sent to the victim, it executes in his/her browser. For successful attack, the victim has to type himself/herself XSS payload in the vulnerable input box and click "Go!" button. That is what I referred as **"Self-Only" XSS**.

Exploitation

During the discussion with Lavakumar, he suggested to check for possible Clickjacking with HTML5 Drag and Drop exploit.

The target page was vulnerable to clickjacking and after spending few hours, I was able to craft a working POC for this attack. Here is the scenario and details:

An attacker hosts a Drag and Drop game which convince the victim to perform Drag and Drop operations. The game page renders the vulnerable Google Code page in an invisible iframe. It also has an element link this:

<div draggable="true" separator" style="clear: both; text-align: center;">[[image: image]]
(https://lh6.googleusercontent.com/-T5PfdVTYoAk/TYhJDxdKG1I/AAAAAAAAAPA/7-usSY43V60/s1600/poc1.JPG)

The victim drags the text to input field which holds the XSS payload.

[[image: image]](https://lh3.googleusercontent.com/-v45mRsJGgi0/TYhJVxsD_KI/AAAAAAAAAPE/pfRkU_vZka4/s1600/poc2.JPG)

Then he/she clicks on the “Go!!” button.

[[image: image]](https://lh5.googleusercontent.com/-GunomnWCSgE/TYhJkVw3QCI/AAAAAAAAAPI/4NXiB3M15-M/s1600/poc3.JPG)

Bingo!!

The Attack – Cookie Stealing

By changing the ‘Evil Data’ in Drag and Drop element, pointed to the attacker’s cookie grabbing script, a successful cookie stealing attack can be performed.

<h3>DRAG ME!!</h3>

The Reward

Google security team has appreciated my efforts and put me on the Google Security Hall of Fame.

Special thanks to Lavakumar!!

kkotowiczhad explained the same issue really well.

Timeline

Bug discovered:15th Feb 2011 **Bug Reported to vendor:** 21st Feb 2011 **Public Disclosure:** 21st Mar 2011

* *